

LLM-based Source Code Compression via Thresholded Symbol Ranking

Angelo Nardone¹  and Paolo Ferragina^{1,2} 

¹ University of Pisa, Department of Computer Science, Pisa, Italy
`angelo.nardone@phd.unipi.it`

² Scuola Superiore Sant’Anna, Pisa, Italy
`paolo.ferragina@santannapisa.it`

Abstract. We study the problem of lossless compression of source code, motivated by the storage demands of large-scale software archives, such as Software Heritage (<https://www.softwareheritage.org/>). General-purpose compressors (e.g., `zstd`, `bzip2`) offer a good trade-off between compression ratio and speed, but fail to exploit all special regularities inherent in source code. Recent approaches leverage Large Language Models (LLMs) within Shannon’s symbol-ranking framework, relying on a scheme in which the predicted rank can grow arbitrarily. While effective at reducing space, this setting incurs significant throughput degradation, and leaves open the question whether it is necessary to explicitly encode all ranks. In this work, we introduce LLM-based compressors deploying two novel symbol-ranking variants that bound predictions to the top- T ranks ($T = 1$ or 63), with out-of-threshold symbols stored as exceptions and compressed jointly with the rank stream via general-purpose compressors. We conduct the first large-scale evaluation of LLM-based source code compression across 30 LLMs, including general-domain, code-specialized, and quantized models. Our T -bounded approach outperforms prior LLM-based compressors both in compression ratio (up to 37% relative improvement) and compression throughput (40% faster). Compared to general-purpose compressors (e.g., `zstd`, `bzip2`), we obtain up to 82% relative compression gain but at a lower speed, thus offering a new trade-off point in the compression-speed spectrum. We also show that these gains are stronger on source code than on natural language, suggesting an interesting indication, namely that source code exposes regularities captured by LLMs but missed by general-purpose exact-match-based compressors. We conclude by commenting on open problems that offer theoretical and practical avenues of research.

Keywords: Lossless compression · Source code compression · Large language models · Symbol ranking · Algorithms

1 Introduction

Data compression is a cornerstone of modern computing, reducing storage, bandwidth, and memory pressure while improving cache efficiency [29]. This is especially critical for lossless compression, which guarantees exact reconstruction in

domains where fidelity is essential, such as textual data, databases, and source code. The need for efficient lossless source-code compression is amplified by the rapid growth of software repositories: platforms such as GitHub and GitLab now host hundreds of millions of projects, and in 2024 alone GitHub reported more than 100 million new projects and nearly 1 billion contributions [18].

This growth affects large-scale archival infrastructures such as the *Software Heritage Archive (SWH)*, launched by INRIA and UNESCO to collect, preserve, and provide universal access to publicly available source code – an effort essential for long-term archival, reproducibility, and for providing a reliable codebase for AI model training. While efficient methods exist for compressing the Merkle DAG underpinning SWH’s infrastructure [17,27], the main challenge lies in compressing the source-code files themselves – commonly referred to as *blobs*. These constitute the overwhelming majority of SWH’s storage footprint and are currently compressed independently with `gzip`-based tools, a strategy that fails to capture redundancies across files. Several approaches have been proposed to address this limitation: [6] showed that compressing files jointly via proper ordering and large-context-window compressors can partially exploit such redundancies, yet at the cost of expensive reorderings and with moderate compression gains.

These limitations motivate new approaches to source-code compression. Since LLMs are effective at modeling sequences, capturing long-range dependencies, and predicting upcoming tokens [12], we investigate their use as predictors within Shannon’s *symbol-ranking* framework [33]. Our goal is twofold: to assess whether LLM-based symbol ranking can improve compression ratio while retaining practical throughput, thus bringing such techniques closer to deployment in large-scale archives, and to advocate for lossless compression as a principled, task-agnostic lens to study and compare foundation models.

State of the Art. Lossless compression via symbol ranking has a long history [33]. Early implementations include the Move-to-Front (MTF) compressor [4] and Prediction by Partial Matching (PPM) [9,10,16,22,26]. A major step forward was the Burrows-Wheeler Transform (BWT) [8], refined by Fenwick [15], which, with MTF and RLE, underpins widely used compressors (e.g., `bzip2` [32]).

The connection between language modeling and compression has long been established [24,31], and the rise of neural networks has revitalized this perspective. Early neural compressors, such as *DeepZip* [19] and *NNCP* [3], combined RNN/LSTM-based prediction with Arithmetic Coding, achieving interesting compression ratios at very high computational cost. With the advent of Transformers [36], LLM-based approaches emerged as strong candidates for lossless compression. *LLMZip* [35] was among the first to exploit LLMs for compression, considering both Symbol Ranking, combined with `zlib`, and Arithmetic Coding driven by LLM-predicted probability distributions. *FineZip* [25] explicitly builds on *LLMZip*, adding parameter-efficient fine-tuning (LoRA) and dynamic context windows; consistently with *LLMZip*, it considers both Symbol Ranking with `bzip2` and Arithmetic Coding, with the latter slightly improving compression but being much slower. *AlphaZip* [28] focused on smaller models to improve throughput, pairing Symbol Ranking with `gzip` or Brotli. Finally, Delé-

tang et al. [12] conducted a broader study of foundation models of different sizes – Chinchilla (70B, 7B, 1B), LLaMA 2 (7B), and vanilla Transformers (3.2M, 800K, 200K) – combined with Arithmetic Coding, showing that they can act as effective compressors for diverse data types, including English text, images and videos, while highlighting trade-offs involving model size, context length, tokenization, and computational cost.

All these works share a common principle: leveraging a (neural) predictor to assign ranks by decreasing predicted likelihood and encoding them via a general-purpose compressor [1,13,30,32], or directly feeding predicted probabilities into Arithmetic Coding [37]. In both cases, every symbol is assigned a rank or probability, corresponding to Shannon’s unbounded setting ($T = \infty$) – no threshold is applied to separate high-confidence predictions from rare symbols. While surpassing general-purpose compressors in compression ratio, both approaches suffer from very low throughput, preventing their use in real-world scenarios, and leave open the question about the impact of T on the final compression ratio.

Our Contribution. In this work, we investigate LLM-based lossless compression for source code, with the following key contributions.

- *Novel symbol-ranking pipelines.* We revisit *symbol ranking for text compression*, introduced by Shannon in his 1951 work [33], and employ LLMs to generate context-driven rankings of the top- T most probable symbols. Shannon formalized three prediction schemes: (i) guessing only the most likely symbol ($T = 1$), with a yes/no outcome and symbol revelation upon error; (ii) scanning the full ranked list until the correct symbol is found ($T = \infty$); and (iii) a thresholded variant that stops after T incorrect guesses and reveals the symbol. We revisit this *thresholded* scheme and propose LLM-based variants focusing on two configurations ($T = 1$ and $T = 63$), comparing them against the unbounded setting ($T = \infty$) of prior work [12,25,28,35] and showing that thresholding enables relevant trade-offs between compression ratio and throughput.
- *Comprehensive model evaluation.* We benchmark 30 LLMs – including code-specialized, general-domain, and quantized models, ranging from 120M to 3B parameters – assessing predictive quality, compression performance, and runtime in the most extensive study to date. For each model, we analyze parameter count, memory footprint, and the impact of code specialization and quantization. Our framework reveals broader trends absent from prior work, which considered at most 3 models of largely varying sizes: specialized code models achieve the best compression, while smaller or quantized models optimize throughput. We also shed new light on scaling laws [21]: larger models consistently improve compression ratio, but at the cost of proportionally lower throughput, highlighting a fundamental trade-off between model capacity and practical efficiency.
- *Measured gains.* Our T -bounded strategies achieve up to 37% relative compression gain and 40% faster throughput over the unbounded approach ($T = \infty$) adopted in prior work [25,28,35], and up to 68% relative compression gain over general-purpose compressors on reordered archives [6], rising to 82% when compressing individual files. Although throughput remains lower than that of

general-purpose compressors, our pipeline operates on individual files (unlike [6]), making parallelization trivial and improving random access to compressed files.

– *Source-code compression.* While prior LLM-based compression literature focused exclusively on natural language corpora or images, we are the first to evaluate this approach on source code – a setting of growing importance, as already motivated. This context was previously explored by Boffa et al. [6] using only general-purpose compressors and costly global reorderings over large archives. We show that LLM-based compressors squeeze source code more than natural language, thus suggesting that source code contains regularities that elude general-purpose compressors but are well captured by LLMs.

These findings indicate that LLM-based symbol-ranking combined with thresholding and lightweight or quantized models represent a promising path toward practical and scalable source-code compression, particularly suited for cold backup scenarios where compression ratio takes priority over speed, paving the way for new pipelines, LLM-specialized models, and novel strategies to reduce costly LLM-inference calls – challenges that we discuss at the end of the paper.

2 Theoretical Background

General-purpose compressors. We use three general-purpose lossless compressors, both as baselines and as backends in our LLM-based pipelines. They cover two major and complementary families: *dictionary-based* compression algorithms, represented by `zstd`, and *block-sorting* compression algorithms, represented by `bzip2` and `bzip3`. We exclude statistical encoders (e.g., Arithmetic Coding) since prior LLM-based compression studies [25,35] show that they substantially increase runtime, reducing overall throughput.

`zstd` extends the LZ77 paradigm with entropy coding and offers several compression levels, trading speed for compression ratio [11,14]. The `bzip` family instead relies on the Burrows–Wheeler Transform, followed by Move-to-Front, Run-Length Encoding, and entropy coding [8,32]. While `bzip2` is the classical representative, `bzip3` introduces several improvements that typically yield stronger compression while preserving competitive decompression speed [34].

Symbol-Ranking Text Compression. The idea of *symbol-ranking text compression* originates from Shannon’s seminal 1951 paper on the entropy of English text [33]. In his experiments, human subjects were asked to predict the next character of a sequence, producing a ranked list of symbols according to their likelihood. Shannon formalized three prediction methods: (i) guessing only the most likely symbol ($T = 1$), receiving a yes/no feedback and, in case of error, returning also the correct symbol; (ii) continuing to guess the correct symbol by scanning their ranked list (by decreasing likelihood), until it is found, thereby recording its rank position ($T = \infty$); and (iii) a hybrid approach, where guessing proceeds as in the previous method but stops after a fixed threshold T of wrong guesses, at which point the correct symbol is revealed.

Starting from these three variants introduced by Shannon, we can formally define the symbol ranking method.

Definition 1 (Symbol Ranking). Let $S = (s_1, \dots, s_n)$ be a string over an alphabet Σ . For each position i , let W_i denote its context (for example, the last k symbols). A predictor associates with W_i a probability distribution over Σ , yielding an ordered list $\Sigma_{W_i} = (\sigma_0, \dots, \sigma_{|\Sigma|-1})$, where $\mathbb{P}(\sigma_0 | W_i) \geq \dots \geq \mathbb{P}(\sigma_{|\Sigma|-1} | W_i)$. The rank of the correct next symbol s_i is then defined as

$$\text{Rank}(s_i | W_i) = j \iff s_i = \sigma_j. \quad (1)$$

The symbol ranking transformation maps S into a sequence of integers (ranks) $R = (r_1, \dots, r_n)$, where $r_i = \text{Rank}(s_i | W_i)$.

Definition 2 (Symbol Ranking with Exceptions). Let S and W_i be as above, and let a predictor associate with each W_i a probability distribution over Σ , yielding the ordered list Σ_{W_i} as in Definition 1. Fix a threshold $T \in \mathbb{N}$. For each position i , the output symbol r_i is

$$r_i = \begin{cases} \text{Rank}(s_i | W_i), & \text{if } \text{Rank}(s_i | W_i) \leq T, \\ \text{ESC}, & \text{otherwise,} \end{cases} \quad (2)$$

where $\text{Rank}(s_i | W_i)$ is defined as in Equation (1), and *ESC* is an escape symbol. In the case of *ESC*, the true symbol s_i is stored in a side list E . The output of symbol ranking with exceptions is thus the pair (R, E) , where R is the sequence of ranks/escapes and E is the list of (symbol) exceptions.

Shannon’s three methods derive from the definitions: the first to Definition 2 with $T = 1$, the second to Definition 1 (equivalently, Definition 2 with $T = \infty$), and the third to Definition 2 with any finite $T > 1$.

It is important to stress that symbol ranking is not a compression algorithm per sé, but a *transform* producing a sequence of integers with a skewed distribution. These can then be efficiently encoded using general-purpose compressors (e.g., *zstd*, *bzip2*, or *bzip3*). The effectiveness of the method ultimately depends on the quality of the predictor. In particular, the more *skewed* the predicted distributions are, the more tokens fall into low ranks, which directly corresponds to lower entropy and thus higher compressibility. It goes without saying that the predictor must be *deterministic* to ensure full reversibility of the compression process, and thus preserve the lossless property in the decompression stage.

LLM-Based Predictors and Proposed Pipeline. The connection between language modeling and compression has long been recognized [24,31]. Recent works have reinforced this link, showing that accurate next-token prediction implies efficient compression. Once text is tokenized – with tokens corresponding to the symbols to be compressed – LLMs can generate “*precise*” probability distributions over the next token given a context, making them ideal predictors for symbol-ranking compression. The key advantage of LLMs resides in the fact that, thanks to neural-network processing and the more recent Attention Mechanism [36], the *k-long context* used to estimate symbol probabilities is not matched *exactly*, as in PPM-like approaches (cfr. [9,10,16,22,26]), but may be *fuzzily and weightily matched*, thus offering a much higher precision in the estimate.

In the Introduction, we discussed early neural approaches. All these works – whether they explicitly adopt a symbol-ranking pipeline [25,28,35] or feed predicted probabilities into a statistical compressor [12,25,35] – rely solely on the approach described in Definition 1 (i.e., $T = \infty$): no threshold is applied to separate high-confidence predictions from rare symbols, leaving unexplored the impact of bounding T on both compression ratio and throughput. These studies consistently show that LLM-based predictors outperform general-purpose compressors in compression ratio, but still suffer from extremely low throughput. Furthermore, most of them evaluate only a single model, and even the most systematic study [12] considers just three models with largely varying sizes, without systematically analyzing which architectural or training factors most influence compression performance across models of comparable scale. Finally, while [12] extends compression experiments to images and video, all these works focus exclusively on classical text corpora, leaving source code – a setting of growing importance, as motivated in the Introduction – entirely unexplored.

Our proposal. We address the limitations of prior work along three complementary directions. First, we extend the symbol-ranking framework by implementing all three of Shannon’s methods (Definitions 1 and 2), introducing for the first time a thresholded pipeline (T -bounded). Second, we benchmark 30 models across architectures and quantization levels on source code – the first large-scale LLM-based compression study on this domain – advocating for compression as a task-agnostic lens to compare foundation models. Third, we show that LLMs capture source-code regularities beyond the reach of general-purpose compressors, achieving compression ratios surpassing those on natural language.

Methodologically, the first pipeline reproduces the unbounded setting ($T = \infty$) of prior work [12,25,28,35]; the second – introduced here – applies a rank threshold T , replacing out-of-threshold tokens with an escape symbol stored in a side list. Both pipelines share a unified architecture optimized for throughput. Each file is processed independently in an *offline setting*, making the pipeline parallelizable across GPUs. Within each file, we use a *dynamic context window* growing up to a fixed maximum M ; longer windows are split with zero stride, ensuring coverage without overlap. Input sequences are processed in batches of size B , enabling parallel rank computation across multiple sequences on GPU.

Finally, the ranked and escaped sequences are compressed using general-purpose lossless compressors – i.e., `zstd`, `bzip2`, and `bzip3` – selected for their balance between compression effectiveness and speed. Statistical encoders such as Arithmetic Coding were deliberately excluded, as prior studies [25,35] showed that their runtime penalties outweigh the modest compression gains.

3 Experimental Setup

All experiments were conducted on an NVIDIA DGX H100 server with $8 \times$ H100 GPUs (80 GB each), dual Intel Xeon Platinum CPUs, and 2 TiB of RAM, running Ubuntu 22.04. The implementation was developed in Python/PyTorch, with

Model	Par.	Qnt.	Code	Model	Par.	Qnt.	Code
StarCoder2-3B (4bit)	3B	Yes	Yes	Llama-3.2-1B (4bit)	1.23B	Yes	No
StarCoder2-3B (8bit)	3B	Yes	Yes	Llama-3.2-1B	1.23B	No	No
StarCoder2-3B (bf16)	3B	No	Yes	Llama-3.2-3B (4bit)	3.21B	Yes	No
StarCoder2-3B (fp32)	3B	No	Yes	Llama-3.2-3B	3.21B	No	No
DeepSeekCoder-1.3B	1.3B	No	Yes	Llama-3.1-8B (4bit)	8B	Yes	No
DeepSeekCoder-1.3B AWQ	1.3B	Yes	Yes	Llama-3.1-8B	8B	No	No
CodeGemma-2B	2.5B	No	Yes	Mistral-7B	7B	No	No
CodeGemma-2B AWQ	2.5B	Yes	Yes	Mistral-7B (4bit)	7B	Yes	No
Gemma-2-2B	2.6B	No	No	Qwen3-1.7B	1.7B	No	No
Gemma-2-2B (4bit)	2.6B	Yes	No	UniXcoder-base	0.13B	No	Yes
SmolLM3-3B	3B	No	No	CodeT5-base	0.22B	No	Yes
Granite-2B	2B	No	No	GPT-2 small	0.124B	No	No
Granite-Code-3B	3B	No	Yes	GPT-2 medium	0.355B	No	No
Granite-Code-3B (4bit)	3B	Yes	Yes	GPT-2 large	0.774B	No	No
Phi-2	2.7B	No	No	GPT-2 xl	1.5B	No	No

Table 1: Evaluated models. *Qnt* denotes quantized models; *Code* denotes code-specialized models. Bold entries denote those selected for the in-depth evaluation.

pretrained models accessed via the Hugging Face ecosystem. This setup enabled medium/large-scale LLMs to be tested efficiently under realistic conditions.

Datasets. We conduct our experiments on *source-code corpora* derived from the Stack-Edu collection [2,20], curated from The Stack v2 [5,23], namely the StarCoder2 training corpus. Stack-Edu provides *SWH identifiers* (SWHIDs) enabling us to fetch the corresponding files directly from the Software Heritage archive. This choice is motivated by three main reasons: (i) it ensures that our experiments are conducted on code that already forms part of the Software Heritage Archive, offering a realistic scenario aligned with our goal of designing compression algorithms for this infrastructure; (ii) it guarantees high-quality code aligned with the training corpora of modern LLMs; and (iii) it provides full reproducibility, since the data are publicly accessible.

Two-phase evaluation strategy. To avoid long experiments, we adopt a strategy based on a proper dataset design (see Section 4), as described below:

– In *Phase One* (exploratory analysis), we construct a ~ 60 MB mixed-language dataset by uniformly sampling ~ 10 MB of source code for each of six representative programming languages: Python, C, C++, C#, Java, and JavaScript. This smaller dataset allows us to evaluate all 30 LLMs in terms of both predictive accuracy and runtime efficiency, without the computational burden of large-scale experiments. The goal is to identify promising models while discarding those that show poor accuracy or excessive runtime.

– In *Phase Two* (in-depth analysis), we focus only on the most promising models and therefore build larger, per-language datasets of approximately ~ 100 MB each, for the same six programming languages used in Phase One, amounting to nearly ~ 600 MB in total. This design enables a detailed language-specific evaluation of the complete compression pipeline. To ensure comparability across datasets, the average file size was constrained to approximately 3KB per file.

Models. The choice of prediction model is critical, as it directly affects both rank accuracy and runtime performance. Except for CodeT5, all models considered are of the decoder-only type. Decoder-only architectures are naturally suited to our

task, as they are designed for autoregressive generation: given a context, they estimate the probability distribution of the next token – the same required for rank computation. Encoder-only and encoder-decoder models are less effective for prediction and were thus only marginally included.

Most tested models fall in the small/mid-scale range (1–3B parameters), reflecting practical runtime constraints. Two larger representatives, **LLaMA-3.1-8B** and **Mistral-7B-v0.3**, are included to cover widely adopted LLM families. Overall, we evaluate 30 models, many considered here for the first time in a lossless compression setting. A schematic summary is provided in Table 1, reporting parameter count and code specialization or quantization. For several families, multiple variants are included (e.g., different precisions or code-specific fine-tuning), allowing us to assess how these design choices affect both compression performance and the broader predictive capabilities of the models.

4 Experimental Results

In this section, we follow the two-phase evaluation strategy mentioned above.

4.1 Phase One (Exploratory Analysis)

In this first phase, we evaluate all 30 LLMs on a 60 MB multilingual code dataset (see Section 3), focusing on predictive accuracy – captured by the rank distribution – and throughput, without performing actual compression. All models use a unified pipeline ($T = \infty$, Definition 1) with fixed $B = 32$ and $M = 512$, selected through preliminary experiments as the configuration maximizing GPU efficiency within memory constraints.

Prediction Results. From a predictive standpoint, most models exhibit highly skewed rank distributions: rank 0 accounts for about 70–80% of occurrences, while the first 63 ranks cover 95–98%. In particular, code-specialized LLMs such as **StarCoder2**, **DeepSeekCoder**, **GraniteCode**, and **CodeGemma** achieve the skewest rank profiles. Among all evaluated models, two clear outliers emerge: **CodeT5**, whose encoder-decoder architecture results in very poor rank accuracy (cfr. Figure 1(a)), and **UniXcoder** together with the **GPT-2** family, which are the only other models falling below the ranges reported above (i.e., 70–80% and 95–98%). In particular, the **GPT-2** family exhibits poor results across all evaluated metrics (e.g., number of tokens, mean, variance), whereas **UniXcoder** maintains values comparable to those of other models. Consequently, **CodeT5** and all **GPT-2** variants are excluded from the following in-depth experimental Phase Two, while **UniXcoder** is retained as a lightweight baseline.

Time Results. Execution-time measurements confirm that throughput constitutes the main bottleneck. Even the fastest models reach at most ~ 0.6 MB/s, while the majority cluster around or below 0.1 MB/s. A negative correlation emerges between GPU memory footprint and throughput (Figures 1(b)): larger models, such as **Llama-8B** and **Mistral**, exhibit slower processing rates (i.e., they occur at the rightmost corner). To balance predictive accuracy and speed, we

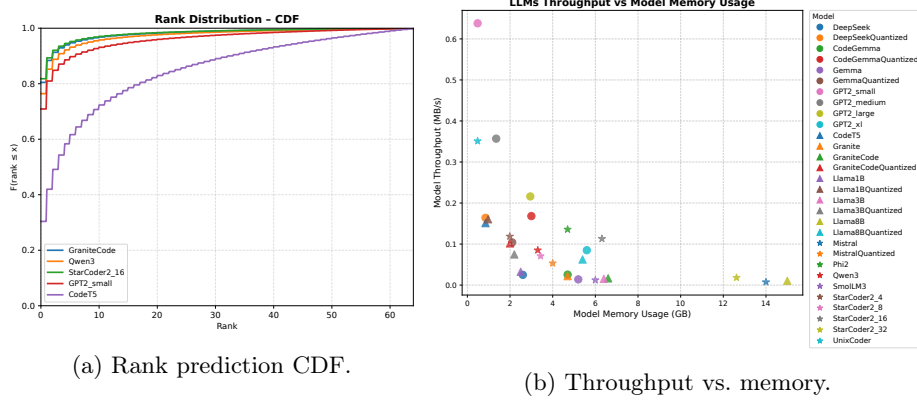


Fig. 1: Exploratory analysis. (a) CDF of rank predictions for models, highlighting the weak predictive accuracy of **CodeT5**. (b) Throughput versus memory usage, showing the inverse relation between model size and inference speed.

define a selection threshold of 0.1 MB/s and retain only those models that satisfy this throughput constraint and the predictive-quality filter discussed above.

Conclusions. The first-phase analysis sheds light on scaling laws [21]: larger models improve prediction quality, but reduce throughput, exposing a capacity–efficiency trade-off. The nine selected LLMs – highlighted in bold in Table 1 – are predominantly quantized, with **StarCoder2-3B (bf16)** as the non-quantized exception. They form the basis of the second experimental phase, where we evaluate the full compression pipeline and the proposed thresholded strategies.

4.2 Phase Two (In-Depth Analysis)

In this second phase, we move to a complete compression evaluation – focusing on compression ratio and throughput – across six language-specific datasets, each containing 100 MB (see Section 3). We first benchmark general-purpose compressors (**zstd**, **bzip2**, **bzip3**) directly on the datasets as baselines, then integrate them within the two symbol-ranking pipelines of Section 2.

Methodology. As baselines, we apply **zstd**, **bzip2**, and **bzip3** without LLM involvement, under both *fast* and *high-compression* settings. Each compressor is tested in two configurations: (i) files compressed independently; (ii) files re-ordered by inverted path names and aggregated into a **.tar** archive, following [6].

For the symbol-ranking pipelines, we keep the same configuration used in Phase One, namely $B = 32$ and $M = 512$. The known pipeline (Definition 1, $T = \infty$) records each token’s exact rank as a 32-bit integer and compresses the resulting stream with the baseline compressors. The new pipeline (Definition 2) instantiates Shannon’s first method with $T = 1$ and his third method with $T \in \{3, 7, 15, 31, 63, 127, 255\}$. Ranks are encoded in $\lceil \log_2(T + 1) \rceil$ bits, while exceptions are stored separately as 32-bit integers; both streams are then com-

pressed with the same baseline compressors. Our experiments identify $T = 63$ as the best configuration for Shannon’s third method, which we adopt as reference.

Results on General-Purpose Compression Baselines. Unless otherwise stated, baseline results refer to the archive-based configuration, which achieves the best compression. As expected, `zstd-3` delivers the highest throughput (≈ 40 MB/s) but also the worst compression, while higher compression level (i.e., `zstd-12` and `zstd-22`) improves compression ratio at the cost of speed. `bzip2` shows nearly identical throughput across the various compression levels, with minor gains in compression for `bzip2-9`. In contrast, `bzip3` offers the best speed-compression trade-off and is therefore adopted as the strongest baseline.

LLM Pipeline Comparison. Across all pipelines, models, and datasets, the execution-time breakdown shows that *rank computation* (i.e., the LLM forward pass), together with tokenization, is the dominant bottleneck, accounting for more than 99% of the total runtime. As a consequence, throughput is determined by the choice of the model rather than by the downstream compressor, and we can focus on the configurations that achieve the best compression.

Our second pipeline, which introduces a rank threshold T , improves over the conventional unbounded approach of the first pipeline ($T = \infty$). With $T = 1$ (Shannon’s first method), throughput improves by up to 40%, at the cost of a small compression loss. In contrast, $T = 63$ (Shannon’s third method) achieves both higher throughput and better compression than $T = \infty$. This is due to two factors: (i) more than 95% of tokens fall within the top-63 predictions, keeping the exception list small, and (ii) ranks require only 6 bits instead of 32. Overall, the $T = 63$ pipeline yields up to 37% additional compression gain and up to 40% throughput improvement over $T = \infty$ across configurations – the latter matching $T = 1$, since extracting top-1 and top-63 predictions is practically time equivalent. These results identify $T = 63$, combined with `bzip3-default` or `zstd-22`, as the most effective LLM-based configuration, which we adopt for the final comparison against general-purpose compressors.

From the model perspective, consistent patterns emerge across all configurations. As expected, the `StarCoder` family achieves the strongest compression, while the smallest model, `UniXcoder`, maximizes throughput at the cost of weaker compression. Models around 1B parameters – in particular `Llama3.2-1B` and `DeepSeekCoder` – offer the best speed-compression trade-off; remarkably without requiring code specialization in the case of `Llama3.2-1B`.

LLM Pipelines vs. General-Purpose Compressors. Across all experiments, LLM pipelines compress better than general-purpose compressors, but with significantly lower throughput. Figure 2 illustrates this trade-off by comparing our best configuration – the $T = 63$ pipeline with `bzip3-default` and representative LLMs – against strong general-purpose compressors in the tar-based setting. In all datasets, the two families occupy distinct regions in the throughput-compression plane: general-purpose compressors cluster in the *upper-right*, offering very high throughput but weaker compression, whereas LLM-based pipelines lie in the *lower-left*, achieving better compression at cost of one–two orders of magnitude lower speed.

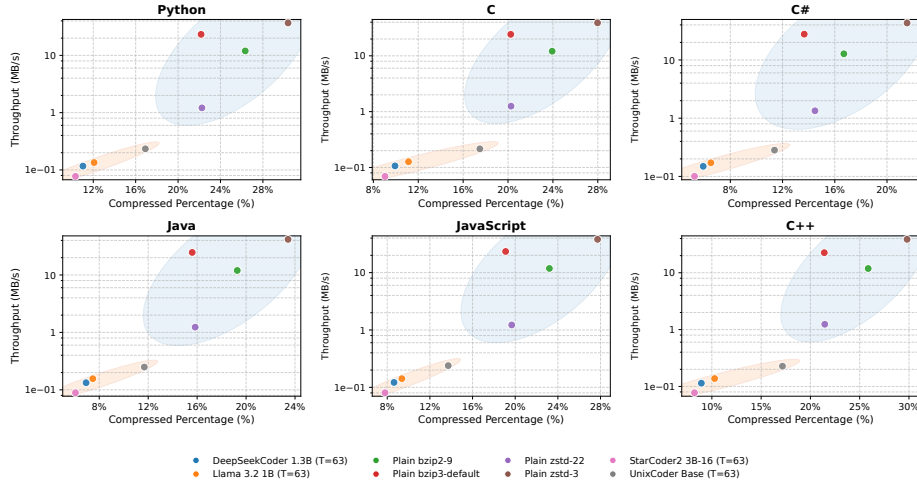


Fig. 2: Throughput (log scale) versus compressed percentage for the best LLM-based pipeline ($T = 63$ with `bzip3-default`) and general-purpose lossless compressors (`zstd-22`, `zstd-3`, `bzip-9`, `bzip3`) in the tar-based setting.

Quantitatively, the best LLM configuration reduces compressed size by up to 68% compared to the strongest baselines. This improvement is significant because our LLM-based method operates on single files (or even partial files, depending on the context window), which makes parallelization trivial. In contrast, general-purpose compressors exploit inter-file similarities by first sorting them (by reversed filename, as in [6]) and applying them with large context windows: hence they incur the cost of computing those reorderings for large source-code collections, such as in Software Heritage. When general-purpose compressors are instead applied to individual files, the best LLM configuration reduces compressed size by up to 82%. Notably, under both baselines, even `UniXcoder`, which yields the weakest compression among our selected LLMs, still outperforms the strongest general-purpose compressors (see Figures 2). The remaining model-level trends are consistent with the previous analysis.

As far as the decompression throughput is concerned, we mention that it is slower than the compression one. This indicates that current LLM-based symbol-ranking schemes still require throughput improvements before becoming competitive for real-world applications. Nonetheless, our results point to a promising direction, especially when combined with lighter architectures or new schemes that reduce costly LLM predictions, also considering that our pipelines can be parallelized across batches. Further future directions are discussed in Section 5.

Text Compression vs. Source Code Compression. As a final comparison, Table 2 contrasts the gains obtained by LLM-based compressors on English text and source code. For text, we use the raw compression results reported by Delétang et al. [12] on `enwik9` with Transformer 800K, Chinchilla-1B, and LLaMA 2 (see Table 1 of [12]), and normalize them against `zstd-22` for a fair comparison. For source code, we report our average gains over the six language-specific

Work	Data	Method	Model	Params	Mode	Gain
Delétang et al. [12]	English text	Arithmetic coding	Transformer-like	800K	one file	$0.99\times$
			Chinchilla	1B	one file	$1.90\times$
			Llama2	7B	one file	$2.59\times$
This work	Source code	Symbol ranking (zstd-22)	UniXcoder	0.13B	collection	$1.89\times$
			Llama-3.2-4bit	1B	collection	$3.04\times$
			DeepSeekCoder-AWQ	1.3B	collection	$3.29\times$
			UniXcoder	0.13B	sorted coll.	$1.18\times$
			Llama-3.2-4bit	1B	sorted coll.	$1.90\times$
			DeepSeekCoder-AWQ	1.3B	sorted coll.	$2.35\times$

Table 2: Compression gains of LLM-based compressors on English text [12] and source code (this work), relative to **zstd-22**. Our results are averaged across six programming languages.

datasets using **UniXcoder**, **Llama3.2-1B**, and **DeepSeekCoder**, again normalized against **zstd-22**. We consider the two scenarios discussed above: files compressed individually, or reordered by inverted path names and packed with **tar** [6].

It is easy to observe that LLM-based compression yields larger gains on source code than on English text. It is striking that at comparable model scales, source-code gains are consistently higher: **DeepSeekCoder** nearly matches the gain of **LLaMA 2** despite being much smaller, and the advantage becomes even larger when source-code files are compressed individually. These results suggest that source code exposes structural regularities that are well captured by LLMs but less effectively exploited by general-purpose compressors such as **zstd-22**.

5 Conclusions and Future Works

Building on Shannon’s symbol-ranking framework, we conducted the first systematic study of LLM-based compression on source-code corpora, and proposed new approaches that yield relative compression gains up to 37% and throughput improvements up to 40% over existing LLM-based methods, while also surpassing general-purpose compressors by up to 82% in relative compression gain, yet remaining slower. Our results show that LLM-based compressors are especially effective on source code, achieving larger gains than on natural language.

Future Works. Despite these advances, throughput remains the limitation of LLM-based compressors. Future work can be divided into two directions. The first concerns pipeline refinements, aimed at reducing the cost of LLM inference: (i) *parsing-based preprocessing* (e.g., Prefix-Free Parsing [7]) to eliminate redundancy before rank computation, and thus reducing the number of LLM-inference steps; (ii) *skipping the first tokens*, where prediction is known to be inaccurate; (iii) dedicated handling of *flat probability distributions*, where rank computation offers little benefit; and (iv) *parallel processing* of batches to improve throughput.

Another direction concerns the extension to *larger datasets* and the development of specialized models for compression: (i) *distillation* of large into lightweight models; (ii) *hardware-aware optimizations*, such as bfloat16 inference or quantization.

References

1. Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A general-purpose data compressor. *ACM Transactions on Information Systems (TOIS)* **37**(1), 1–30 (2018)
2. Allal, L.B., Lozhkov, A., Bakouch, E., Blázquez, G.M., Penedo, G., et al.: Smol-LLM2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737* (2025)
3. Bellard, F.: Lossless data compression with neural networks (2019), available at <https://bellard.org/nncp/nncp.pdf>
4. Bentley, J.L., Sleator, D.D., Tarjan, R.E., Wei, V.K.: A locally adaptive data compression scheme. *Communications of the ACM* **29**(4), 320–330 (1986)
5. BigCode: The stack v2 (2024), software repository at Hugging Face: <https://huggingface.co/datasets/bigcode/the-stack-v2>
6. Boffa, A., et al.: On the compressibility of large-scale source code datasets. *Journal of Systems and Software* **227**, 112429 (2025)
7. Boucher, C., Gague, T., Kuhnle, A., Langmead, B., Manzini, G., Mun, T.: Prefix-free parsing for building big bwts. *Algorithms for Molecular Biology* **14**(1), 13 (2019)
8. Burrows, M., Wheeler, D.J.: A block-sorting lossless data compression algorithm. Tech. rep., Digital Equipment Corporation, Systems Research Center (1994)
9. Cleary, J.G., Teahan, W.J.: Unbounded length contexts for PPM. *The Computer Journal* **40**(2–3), 67–75 (1997)
10. Cleary, J.G., Witten, I.H.: Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications* **32**(4), 396–402 (1984)
11. Collet, Y.: Zstandard compression and the application/zstd media type (2021), rFC 8878, Internet Engineering Task Force
12. Delétang, G., Ruoss, A., Duquenne, P.A., Catt, E., Genewein, T., Mattern, C., Grau-Moya, J., Wenliang, L.K., Aitchison, M., Orseau, L., et al.: Language modeling is compression. In: *International Conference on Learning Representations*. vol. 2024, pp. 14165–14181 (2024)
13. Deutsch, P., Gailly, J.L.: Rfc1950: Zlib compressed data format specification version 3.3 (1996)
14. Facebook: Zstandard (Zstd): Fast real-time compression algorithm (2025), software repository
15. Fenwick, P.: Block sorting text compression – final report. Tech. rep., University of Auckland (1996)
16. Ferragina, P.: *Pearls of Algorithm Engineering*. Cambridge University Press (2023)
17. Fontana, T., Vigna, S., Zacchiroli, S.: Webgraph: The next generation (is in rust). In: *Companion Proceedings of the ACM Web Conference*. pp. 686–689 (2024)
18. GitHub: News and insights (2025), available at <https://github.blog/news-insights/>. Accessed: 2025-10-28
19. Goyal, M., Tatwawadi, K., Chandak, S., Ochoa, I.: Deepzip: Lossless data compression using recurrent neural networks. *arXiv preprint arXiv:1811.08162* (2018)
20. HuggingFaceTB: Stack-edu dataset (2025), dataset available via Hugging Face
21. Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020)
22. Lelewer, D.A., Hirschberg, D.S.: Streamlining context models for data compression. In: *Proceedings of the Data Compression Conference*. p. 91 (1991)

23. Lozhkov, A., Li, R., Allal, L.B., Cassano, F., Lamy-Poirier, J., Tazi, N., Tang, A., Pykhtar, D., Liu, J., Wei, Y., et al.: StarCoder 2 and The Stack v2: The next generation. arXiv preprint arXiv:2402.19173 (2024)
24. Mahoney, M.V.: Fast text compression with neural networks. In: Proceedings of the 13th International Florida Artificial Intelligence Research Society Conference (FLAIRS). pp. 230–234 (2000)
25. Mittu, F., Bu, Y., Gupta, A., Devireddy, A., Ozdarendeli, A.E., Singh, A., Anumanchipalli, G.: Finezip: Pushing the limits of large language models for practical lossless text compression. arXiv preprint arXiv:2409.17141 (2024)
26. Moffat, A.: Implementing the PPM data compression scheme. *IEEE Transactions on Communications* **38**(11), 1917–1921 (1990)
27. Morris, B.: The components of the wired spanning forest are recurrent. *Probability Theory and Related Fields* **125**(2), 259–265 (2003)
28. Narashiman, S.S., Chandrachoodan, N.: Alphazip: Neural network-enhanced lossless text compression. arXiv preprint arXiv:2409.15046 (2024)
29. Navarro, G.: Compact data structures: a practical approach. Cambridge University Press (2016)
30. Peter, D.: Gzip file format specification version 4.3. Request for Comments: 1952 (1996)
31. Schmidhuber, J., Heil, S.: Sequential neural text compression. *IEEE Transactions on Neural Networks* **7**(1), 142–146 (1996)
32. Seward, J.: bzip2 and libbzip2: A program and library for data compression (2000), available at <https://sourceware.org/pub/bzip2/docs/manual.pdf>. Accessed: 2025-10-28
33. Shannon, C.E.: Prediction and entropy of printed english. *Bell System Technical Journal* **30**(1), 50–64 (1951)
34. Szewczyk, K.: bzip3: A better, faster and stronger spiritual successor to BZip2 (2022), available at <https://github.com/kspalaiologos/bzip3>. Accessed: 2025-10-28
35. Valmeekam, C.S.K., Narayanan, K., Kalathil, D., Chamberland, J.F., Shakkottai, S.: LLMzip: Lossless text compression using large language models. arXiv preprint arXiv:2306.04050 (2023)
36. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*. vol. 30 (2017)
37. Witten, I.H., Neal, R.M., Cleary, J.G.: Arithmetic coding for data compression. *Communications of the ACM* **30**(6), 520–540 (1987)